PROGRAMACIÓN DIDÁCTICA: PROGRAMACIÓN (0485)

Resumen de Unidades de Programación (Curso 2025-2026)

Código	Título de la UP	Duración	Temporalización	RA asociados (Descripción breve)
UP01	**Primeras líneas**	18 h	08/09/25 – 22/09/25	**RA01:** Estructura de un programa. Variables, tipos, operadores y comentarios
UP02	**El flujo del programa**	26 h	23/09/25 – 20/10/25	**RA03 (parcial):** Estructuras de control. Selección, repetición y salto
UP03	**El mundo de los objetos**	28 h	21/10/25 – 19/11/25	**RA02:** Fundamentos de POO. Clases, objetos, métodos, constructores
UP04	**Clases en acción**	46 h	20/11/25 – 21/01/26	**RA04:** POO avanzada. Herencia, interfaces, polimorfismo **RA06 (parcial):** Arrays y tipos avanzados

Código	Título de la UP	Duración	Temporalización	RA asociados (Descripción breve)
UP05	**Datos que persisten**	48 h	22/01/26 – 04/03/26	**RA05:** E/S consola, ficheros e interfaces gráficas **RA03 (resto):** Excepciones
UP06	**Arquitectura de clases**	44 h	05/03/26 – 15/04/26	**RA07:** Herencia avanzada, clases abstractas e interfaces **RA06 (resto):** Colecciones, genéricos y expresiones regulares
UP07	**Datos en la base**	56 h	16/04/26 – 06/05/26	**RA08:** BBDD orientadas a objetos **RA09:** Acceso a BBDD relacionales con JDBC

- La suma total de estas unidades es de **266 horas**, correspondientes íntegramente al periodo de formación realizado en el centro educativo fuera del periodo dual.

UP01: PRIMERAS LÍNEAS

1. Identificación

Campo	Detalle
Código	UP01
Módulo	Programación (0485)
Duración	**18 Horas**
Temporalización	Del **08/09/2025** al **22/09/2025**

2. Fundamentación

Resultado de Aprendizaje * **RA01.** Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

Criterios de Evaluación * **a)** Se han identificado los bloques que componen la estructura de un programa informático. * **b)** Se han creado proyectos de desarrollo de aplicaciones. * **c)** Se han utilizado entornos integrados de desarrollo. * **d)** Se han identificado los distintos tipos de variables y la utilidad específica de cada uno. * **e)** Se ha modificado el código de un programa para crear y utilizar variables. * **f)** Se han creado y utilizado constantes y literales. * **g)** Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje. * **h)** Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas. * **i)** Se han introducido comentarios en el código.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Introducción a la programación: algoritmos, programas y lenguajes de programación. * Clasificación de los lenguajes: alto/bajo nivel, interpretados/compilados, paradigmas. * Entornos de desarrollo integrados (IDE): elementos y uso del IDE (IntelliJ IDEA / VS Code). * El JDK: JVM, JRE, compilador `javac` y empaquetador `jar`. * Proyectos Java: estructura de carpetas, paquetes (`package`) e `import`. * Variables: declaración, tipos primitivos (`int`, `double`, `boolean`, `char`) y `String`. * Literales y constantes (`final`). * Operadores: aritméticos, relacionales y lógicos. Expresiones. * Conversiones de tipo: automáticas y *casting* explícito. * Comentarios: de línea (`//`), de bloque (`/* */`) y JavaDoc (`/** */`). * Normas y estilo de codificación: nomenclatura, indentación, separaciones y organización de ficheros.

Metodología * Metodologías activas centradas en el alumnado. El aprendizaje se organiza en torno a **prácticas guiadas de programación** donde el estudiante construye, de forma incremental, pequeños programas Java usando el IDE, descubriendo los conceptos a medida que los necesita.

Secuencia de actividades * **A1:** Introducción a la programación y al IDE. Creación del primer proyecto Java (“Hola Mundo”). Identificación de la estructura básica de un programa. * **A2:** Variables y tipos de datos. Declaración, asignación y uso de variables de diferentes tipos en un programa sencillo. * **A3:** Constantes, literales y operadores. Escritura de expresiones con operadores aritméticos, relacionales y lógicos. * **A4:** Conversiones de tipo (automáticas y *casting*) y comentarios JavaDoc. Documentación del código desarrollado.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA01. * **Rúbricas de prácticas (20%):** valoración sistemática de los programas desarrollados en cada actividad, atendiendo a la corrección técnica, el estilo de codificación y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP02: EL FLUJO DEL PROGRAMA

1. Identificación

Campo	Detalle
Código	UP02
Módulo	Programación (0485)
Duración	**26 Horas** (Parte de las 26 h totales del RA03)
Temporalización	Del **23/09/2025** al **20/10/2025**

2. Fundamentación

Resultado de Aprendizaje * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Criterios a, b, c, e, f, g — la gestión de excepciones se completa en UP05)*

Criterios de Evaluación * **a)** Se ha escrito y probado código que haga uso de estructuras de selección. * **b)** Se han utilizado estructuras de repetición. * **c)** Se han reconocido las posibilidades de las sentencias de salto. * **e)** Se han creado programas ejecutables utilizando diferentes estructuras de control. * **f)** Se han probado y depurado los programas. * **g)** Se ha comentado y documentado el código.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Flujos de entrada y salida: concepto de *stream*, `System.in` y `System.out`. * Entrada desde teclado con `Scanner`: `nextInt()`, `nextDouble()`, `nextLine()`. * Salida por pantalla: `print()`, `println()`, `printf()` y `DecimalFormat`. * Programación estructurada: estructuras secuencial, de selección y repetitiva. * Estructuras de selección: `if`, `if-else`, operador ternario `? :`, `if-else if-else` y `switch`. * Estructuras de repetición: `while`, `do-while`, `for` y `for-each`. * Bucles anidados y bucles infinitos. * Estructuras de salto: `break`, `continue` y `return`. * Introducción a los algoritmos: representación mediante diagramas de flujo y pseudocódigo. * Depuración de programas con el depurador del IDE: *breakpoints*, ejecución paso a paso y visualización de variables.

Metodología * Metodologías activas basadas en **prácticas de programación** progresivas. El alumnado resuelve problemas de dificultad creciente implementando y depurando programas con estructuras de control.

Secuencia de actividades * **A1:** Flujos de E/S. Lectura de datos desde teclado y salida formateada por pantalla. * **A2:** Estructuras de selección (`if`, `if-else`, `switch`). Programas de clasificación y toma de decisiones. * **A3:** Estructuras de repetición (`while`, `do-while`, `for`). Bucles simples y anidados. * **A4:** Sentencias de salto (`break`, `continue`) y depuración con el IDE. Algoritmos: diagramas de flujo y pseudocódigo.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA03 trabajados en esta UP. * **Rúbricas de prácticas (20%):** valoración de los programas desarrollados en cada actividad, atendiendo a la corrección lógica, la elección de la estructura de control adecuada y la documentación. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño**

Universal para el Aprendizaje, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP03: EL MUNDO DE LOS OBJETOS

1. Identificación

Campo	Detalle
Código	UP03
Módulo	Programación (0485)
Duración	**28 Horas** (18 h del RA02 + 10 h inicio del RA04)
Temporalización	Del **21/10/2025** al **19/11/2025**

2. Fundamentación

Resultados de Aprendizaje * **RA02.** Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos. * **RA04.** Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos. *(Criterios a–e, inicio)*

Criterios de Evaluación * **RA02. a)** Se han identificado los fundamentos de la programación orientada a objetos. * **RA02. b)** Se han escrito programas simples. * **RA02. c)** Se han instanciado objetos a partir de clases predefinidas. * **RA02. d)** Se han utilizado métodos y propiedades de los objetos. * **RA02. e)** Se han escrito llamadas a métodos estáticos. * **RA02. f)** Se han utilizado parámetros en la llamada a métodos. * **RA02. g)** Se han incorporado y utilizado librerías de objetos. * **RA02. h)** Se han utilizado constructores. * **RA02. i)** Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples. * **RA04. a)** Se ha reconocido la sintaxis, estructura y componentes típicos de una clase. * **RA04. b)** Se han definido clases. * **RA04. c)** Se han definido propiedades y métodos. * **RA04. d)** Se han creado constructores. * **RA04. e)** Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Introducción a la POO: abstracción, clases y objetos. * Atributos de instancia y de clase (`static`). Métodos de instancia y estáticos. * Constructores: constructor por defecto y sobrecarga de constructores. * Encapsulamiento y modificadores de acceso (`public`, `private`, `protected`). * *Getters* y *setters*. * La palabra clave `this`. * Uso de librerías y clases predefinidas de Java (`Math`, `String`, `Integer`, etc.). * Relaciones entre clases: asociación, agregación y composición. * Normas de codificación en POO: nomenclatura de clases, métodos y atributos.

Metodología * Metodologías activas basadas en **prácticas de programación** orientadas a objetos. El alumnado modela objetos del mundo real, define sus clases con atributos y métodos, crea instancias y las utiliza en programas completos.

Secuencia de actividades * **A1:** Introducción a la POO. Abstracción y modelado de clases sencillas con atributos y métodos. Creación del primer objeto. * **A2:** Constructores (por defecto y sobrecargados). Encapsulamiento, *getters* y *setters*. Uso de `this`. * **A3:** Métodos estáticos. Uso de librerías predefinidas (`Math`, `String`). Llamadas a métodos con parámetros. * **A4:** Relaciones entre clases (asociación, agregación, composición). Diseño y desarrollo de un programa con varias clases relacionadas.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA02 y del inicio del RA04. * **Rúbricas de prácticas (20%):** valoración de los programas orientados a objetos desarrollados en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP04: CLASES EN ACCIÓN

1. Identificación

Campo	Detalle
Código	UP04
Módulo	Programación (0485)
Duración	**46 Horas** (26 h del RA04 + 20 h del RA06 parcial)
Temporalización	Del **20/11/2025** al **21/01/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA04.** Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos. *(Criterios f–i, continuación)* * **RA06.** Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos. *(Criterios a–e)*

Criterios de Evaluación * **RA04. f)** Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros. * **RA04. g)** Se han definido y utilizado clases heredadas. * **RA04. h)** Se han creado y utilizado métodos estáticos. * **RA04. i)** Se han creado y utilizado conjuntos y librerías de clases. * **RA06. a)** Se han escrito programas que utilicen matrices (*arrays*). * **RA06. b)** Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados. * **RA06. c)** Se han utilizado listas para almacenar y procesar información. * **RA06. d)** Se han utilizado iteradores para recorrer los elementos de las listas. * **RA06. e)** Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Herencia: superclases y subclases, la palabra clave `extends`. Transititividad. * Modificadores de visibilidad en jerarquías de clases. * Herencia de constructores: `super()` y `this()`. * Sobreescritura de métodos (`@Override`) y sobreescritura del método `toString()`. * Clases abstractas y métodos abstractos. Clases finales y métodos finales. * Interfaces: definición, implementación (`implements`) y diferencias con clases abstractas. * Polimorfismo: sobrecarga y sobreescritura. Polimorfismo en colecciones. * *Arrays* unidimensionales y multidimensionales: declaración, inicialización y recorrido. * Colecciones Java (`java.util`): `ArrayList`, `LinkedList`, `HashMap`. Iteradores. * Características y ventajas de cada tipo de colección. Selección según caso de uso.

Metodología * Metodologías activas basadas en **prácticas de programación**. El alumnado diseña jerarquías de clases, implementa interfaces y desarrolla programas que emplean colecciones para

gestionar conjuntos de objetos.

Secuencia de actividades * **A1:** Herencia simple. Diseño y codificación de jerarquías de clases. Constructores heredados (`super`). * **A2:** Sobreescritura de métodos (`@Override`). Clases abstractas e interfaces. Polimorfismo. * **A3:** *Arrays* unidimensionales y multidimensionales. Recorrido y operaciones básicas. * **A4:** Colecciones (`ArrayList`, `HashMap`). Iteradores. Selección de la colección adecuada según el problema.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA04 (continuación) y del RA06 (parcial). * **Rúbricas de prácticas (20%):** valoración de los programas con jerarquías de clases y colecciones desarrollados en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP05: DATOS QUE PERSISTEN

1. Identificación

Campo	Detalle
Código	UP05
Módulo	Programación (0485)
Duración	**48 Horas** (44 h del RA05 + 4 h resto del RA03: excepciones)
Temporalización	Del **22/01/2026** al **04/03/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA05.** Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases. * **RA03.** Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje. *(Criterios d, h, i — gestión de excepciones y aserciones)*

Criterios de Evaluación * **RA05. a)** Se ha utilizado la consola para realizar operaciones de entrada y salida de información. * **RA05. b)** Se han aplicado formatos en la visualización de la información. * **RA05. c)** Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas. * **RA05. d)** Se han utilizado ficheros para almacenar y recuperar información. * **RA05. e)** Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros. * **RA05. f)** Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples. * **RA05. g)** Se han programado controladores de eventos. * **RA05. h)** Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información. * **RA03. d)** Se ha escrito código utilizando control de excepciones. * **RA03. h)** Se han creado excepciones. * **RA03. i)** Se han utilizado aserciones para la detección y corrección de errores durante la fase de desarrollo.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Concepto de flujo (*stream*): flujos de bytes y de caracteres. Jerarquía `java.io`. * Flujos estándar: `System.in`, `System.out`, `System.err`. * Entrada desde teclado: `Scanner` y sus métodos de lectura. * Salida por pantalla con formato: `printf`, `DecimalFormat`. * Sistema de ficheros: clase `File`. Crear, borrar, renombrar y listar ficheros y directorios. * Lectura y escritura de ficheros de texto: `FileReader`, `BufferedReader`, `FileWriter`, `PrintWriter`. * Lectura y escritura de ficheros binarios: `FileInputStream`, `FileOutputStream`. * Persistencia y serialización: `ObjectOutputStream`, `ObjectInputStream`, interfaz `Serializable`. * Gestión de excepciones: jerarquía `Throwable`. Excepciones *checked* y *unchecked*. * Captura de excepciones: `try-catch-finally`. Excepciones múltiples. * Propagación de excepciones: `throws` y `throw`. * Creación de excepciones propias (subclases de `Exception`). * Aserciones (`assert`) para la detección de errores en fase de desarrollo. * Interfaces gráficas de usuario (GUI) con Java Swing: ventanas, componentes básicos y *layout*. * Controladores de eventos: *listeners* y manejo de eventos de usuario.

Metodología * Metodologías activas basadas en **prácticas de programación**. El alumnado desarrolla aplicaciones que leen y escriben datos desde distintas fuentes (consola, ficheros), gestiona los errores mediante excepciones y construye interfaces gráficas sencillas.

Secuencia de actividades * **A1:** Flujos de E/S. Lectura y escritura de ficheros de texto y binarios. Sistema de ficheros con `File`. * **A2:** Persistencia y serialización de objetos. Lectura y escritura de

objetos en ficheros. * **A3:** Gestión de excepciones: captura (`try-catch-finally`) y propagación (`throws`/`throw`). Creación de excepciones propias. Aserciones. * **A4:** Interfaces gráficas con Java Swing: ventanas, botones, campos de texto y cuadros de diálogo. Controladores de eventos. Programa completo con GUI de entrada y salida.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA05 y de los criterios restantes del RA03. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de E/S, ficheros y GUI desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP06: ARQUITECTURA DE CLASES

1. Identificación

Campo	Detalle
Código	UP06
Módulo	Programación (0485)
Duración	**44 Horas** (20 h del RA07 + 24 h resto del RA06)
Temporalización	Del **05/03/2026** al **15/04/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA07.** Desarrolla programas aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación. * **RA06.** Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos. *(Criterios f–j, continuación)*

Criterios de Evaluación * **RA07. a)** Se han identificado los conceptos de herencia, superclase y subclase. * **RA07. b)** Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos. * **RA07. c)** Se ha reconocido la incidencia de los constructores en la herencia. * **RA07. d)** Se han creado clases heredadas que sobrescriben la implementación de métodos de la superclase. * **RA07. e)** Se han diseñado y aplicado jerarquías de clases. * **RA07. f)** Se han probado y depurado las jerarquías de clases. * **RA07. g)** Se han realizado programas que implementen y utilicen jerarquías de clases. * **RA07. h)** Se ha comentado y documentado el código. * **RA07. i)** Se han identificado y evaluado los escenarios de uso de interfaces. * **RA07. j)** Se han identificado y evaluado los escenarios de utilización de la herencia y la composición. * **RA06. f)** Se han creado clases y métodos genéricos. * **RA06. g)** Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto. * **RA06. h)** Se han identificado las clases relacionadas con el tratamiento de documentos escritos en diferentes lenguajes de intercambio de datos. * **RA06. i)** Se han realizado programas que realicen manipulaciones sobre documentos escritos en diferentes lenguajes de intercambio de datos. * **RA06. j)** Se han utilizado operaciones agregadas para el manejo de información almacenada en colecciones.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos * Consolidación y profundización de herencia: modificadores `final` y `abstract`. Diseño de jerarquías complejas. * Herencia de constructores (`super`) y sobreescritura avanzada (`@Override`). * Herencia frente a composición: criterios de elección según el diseño. * Escenarios de uso de interfaces: simulación de herencia múltiple, contratos de comportamiento. * Genéricos: clases y métodos genéricos (`<T>`). Ventajas respecto a `Object`. * Operaciones agregadas sobre colecciones: *streams* de Java, `filter`, `map`, `reduce`, `forEach`. * Expresiones regulares (`java.util.regex`): patrones, búsqueda y validación en cadenas de texto. * Intercambio de datos: introducción a JSON y XML; clases Java para su manipulación. * Documentación del código con JavaDoc: generación de documentación de un proyecto completo. * Prueba y depuración de jerarquías de clases con el depurador del IDE.

Metodología * Metodologías activas basadas en **prácticas de programación** y análisis de diseño. El alumnado consolida el diseño orientado a objetos, aplica genéricos, usa expresiones regulares y documenta formalmente un proyecto.

Secuencia de actividades * **A1:** Jerarquías de clases avanzadas. Modificadores `final` y `abstract`. Herencia frente a composición. Depuración de jerarquías con el IDE. * **A2:** Interfaces: escenarios de uso. Clases y métodos genéricos. Operaciones agregadas con *streams*. * **A3:** Expresiones regulares: búsqueda y validación de patrones. Tratamiento de datos en formato JSON/XML. * **A4:** Documentación JavaDoc completa de un proyecto. Revisión y depuración final del código.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA07 y de los criterios restantes del RA06. * **Rúbricas de prácticas (20%):** valoración de los programas con jerarquías avanzadas, genéricos y expresiones regulares desarrollados en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar el bloque para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas. —

UP07: DATOS EN LA BASE

1. Identificación

Campo	Detalle
Código	UP07
Módulo	Programación (0485)
Duración	**56 Horas** (24 h del RA08 + 32 h del RA09)
Temporalización	Del **16/04/2026** al **06/05/2026**

2. Fundamentación

Resultados de Aprendizaje * **RA08.** Utiliza bases de datos orientadas a objetos, analizando sus características y aplicando técnicas para mantener la persistencia de la información. * **RA09.** Gestiona información almacenada en bases de datos manteniendo la integridad y consistencia de los datos.

Criterios de Evaluación * **RA08. a)** Se han identificado las características de las bases de datos orientadas a objetos. * **RA08. b)** Se ha analizado su aplicación en el desarrollo de aplicaciones mediante lenguajes orientados a objetos. * **RA08. c)** Se han instalado sistemas gestores de bases de datos orientados a objetos. * **RA08. d)** Se han clasificado y analizado los distintos métodos soportados por los sistemas gestores para la gestión de la información almacenada. * **RA08. e)** Se han creado

bases de datos y las estructuras necesarias para el almacenamiento de objetos. * **RA08. f)** Se han programado aplicaciones que almacenen objetos en las bases de datos creadas. * **RA08. g)** Se han realizado programas para recuperar, actualizar y eliminar objetos de las bases de datos. * **RA08. h)** Se han realizado programas para almacenar y gestionar tipos de datos estructurados, compuestos y relacionados. * **RA09. a)** Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos. * **RA09. b)** Se han programado conexiones con bases de datos. * **RA09. c)** Se ha escrito un código para almacenar información en bases de datos. * **RA09. d)** Se han creado programas para recuperar y mostrar información almacenada en bases de datos. * **RA09. e)** Se han efectuado borrados y modificaciones sobre la información almacenada. * **RA09. f)** Se han creado aplicaciones que muestren la información almacenada en bases de datos. * **RA09. g)** Se han creado aplicaciones para gestionar la información presente en bases de datos.

Competencias * **Profesionales:** e), j), q), w). * **Ocupación:** e), j), t), w).

3. Organización

Contenidos

Bases de datos orientadas a objetos (RA08): * Características de las BBDD orientadas a objetos frente a las relacionales. * Sistemas gestores de BBDD orientadas a objetos: instalación y configuración. * Creación de bases de datos y estructuras de almacenamiento de objetos. * Operaciones CRUD sobre objetos: almacenamiento, recuperación, actualización y eliminación. * Almacenamiento de tipos de datos estructurados, compuestos y relacionados.

Acceso a bases de datos relacionales con JDBC (RA09): * Repaso del modelo relacional y SQL: DDL, DML y TCL. * El driver JDBC: tipos de drivers, instalación y configuración. * Establecimiento y cierre de la conexión (`Connection`, `DriverManager`). * Ejecución de sentencias SQL: `Statement` y `PreparedStatement`. * Recuperación de resultados: objeto `ResultSet` y sus métodos de navegación. * Operaciones CRUD desde Java: INSERT, SELECT, UPDATE y DELETE. * Gestión de transacciones: propiedades ACID, `commit()` y `rollback()`. * Aplicaciones completas con interfaz de usuario que gestionan información en BBDD.

Metodología * Metodologías activas basadas en **prácticas de programación** y un proyecto integrador final. El alumnado desarrolla aplicaciones Java que persisten objetos en bases de datos, tanto orientadas a objetos como relacionales, gestionando el acceso mediante JDBC.

Secuencia de actividades * **A1:** Instalación y configuración del SGBD orientado a objetos. Creación de estructuras de almacenamiento y operaciones CRUD sobre objetos. * **A2:** Tipos de datos estructurados y relacionados en BBDD orientadas a objetos. Programas de almacenamiento y recuperación. * **A3:** Driver JDBC. Conexión a una BBDD relacional (MySQL). Ejecución de sentencias SQL con `Statement` y `PreparedStatement`. * **A4:** Recuperación de resultados (`ResultSet`). Operaciones CRUD

completas. Gestión de transacciones. * **A5:** Aplicación final: programa Java con interfaz de usuario que gestiona información almacenada en una base de datos relacional.

Recursos * Equipos del aula de informática, IDE Java (VS Codium), JDK, SGBD orientado a objetos, servidor MySQL, driver JDBC (`mysql-connector-java`), control de versiones (Git/GitHub) y aula virtual (Aules).

4. Evaluación y adaptación

Instrumentos de evaluación La evaluación será continua y formativa, enfocada en la adquisición de los criterios de evaluación del RA08 y del RA09. * **Rúbricas de prácticas (20%):** valoración de las aplicaciones de acceso a bases de datos desarrolladas en cada actividad. * **Pruebas objetivas (80%):** prueba teórico-práctica al finalizar cada RA para verificar el cumplimiento de los criterios de evaluación.

Adaptaciones * **Medidas según necesidades:** las adaptaciones se aplicarán de manera flexible tras la evaluación inicial y el seguimiento diario del progreso del alumnado. * **DUA:** se empleará el **Diseño Universal para el Aprendizaje**, proporcionando materiales en múltiples formatos y permitiendo la flexibilización de tiempos en las actividades prácticas.